Nile University

Lecture (06)

More About Classes and Object-Oriented Programming 1

Topics

7.14 Introduction to Object-Oriented Analysis and Design
7.15 Screen Control

11.1 The `this` Pointer and Constant Member Functions
11.2 Static Members
11.3 Friends of Classes
11.4 Memberwise Assignment
11.5 Copy Constructors

7.14 Introduction to Object-Oriented Analysis and Design

- Object-Oriented Analysis: that phase of program development when the program functionality is determined from the requirements
- It includes
  - identification of objects and classes
  - definition of each class's attributes
  - identification of each class's behaviors
  - definition of the relationship between classes

Identify Objects and Classes

- Consider the major data elements and the operations on these elements
- Candidates include
  - user-interface components (menus, text boxes, *etc.*)
  - I/O devices
  - physical objects
  - historical data (employee records, transaction logs, *etc.*)
  - the roles of human participants

Define Class Attributes

- Attributes are the data elements of an object of the class
- They are necessary for the object to work in its role in the program

Define Class Behaviors

- For each class,
  - Identify what an object of a class should do in the program
- The behaviors determine some of the member functions of the class

Relationships Between Classes

Possible relationships

- Access ("uses-a")
- Ownership/Composition ("has-a")
- Inheritance ("is-a")

Finding the Classes

Technique:

- Write a description of the problem domain (objects, events, etc. related to the problem)
- List the nouns, noun phrases, and pronouns. These are all candidate objects
- Refine the list to include only those objects that are relevant to the problem

Determine Class Responsibilities

Class responsibilities:

- What is the class responsible to know?
- What is the class responsible to do?

Use these to define some of the member functions

Object Reuse

- A well-defined class can be used to create objects in multiple programs
- By re-using an object definition, program development time is shortened
- One goal of object-oriented programming is to support object reuse

7.15 Screen Control

- Programs to date have all displayed output starting at the upper left corner of computer screen or output window. Output is displayed left-to-right, line-by-line.
- Computer operating systems are designed to allow programs to access any part of the computer screen. Such access is operating system-specific.

Screen Control – Concepts

- An output screen can be thought of as a grid of 25 rows and 80 columns. Row 0 is at the top of the screen. Column 0 is at the left edge of the screen.
- The intersection of a row and a column is a cell. It can display a single character.
- A cell is identified by its row and column number. These are its coordinates.

Screen Control – Windows - Specifics

- `#include <windows.h>` to access the operating system from a program
- Create a handle to reference the output screen:

```
HANDLE screen = GetStdHandle(STD_OUTPUT_HANDLE);
```

- Create a COORD structure to hold the coordinates of a cell on the screen:

```
COORD position;
```

Screen Control – Windows – More Specifics

- Assign coordinates where the output should appear:

```
position.X = 30;    // column
position.Y = 12;    // row
```

- Set the screen cursor to this cell:

```
SetConsoleCursorPosition(screen, position);
```

- Send output to the screen:

```
cout << "Look at me!" << endl;
```

  - be sure to end with **`endl`**, not **`'\n'`** or nothing

11.1 The `this` Pointer and Constant Member Functions

- `this` pointer:
  - Implicit parameter passed to a member function
  - points to the object calling the function
- `const` member function:
  - does not modify its calling object

Using the `this` Pointer

Can be used to access members that may be hidden by parameters with the same name:

```
class SomeClass
{
  private:
    int num;
  public:
    void setNum(int num)
    { this->num = num; }
};
```

Constant Member Functions

- Declared with keyword **`const`**

- When **`const`** appears in the parameter list,

```
int setNum (const int num)
```

the function is prevented from modifying the parameter. The parameter is read-only.

- When **`const`** follows the parameter list,

```
int getX()const
```

the function is prevented from modifying the object.

11.2 Static Members

- Static member variable:
  - One instance of variable for the entire class
  - Shared by all objects of the class
- Static member function:
  - Can be used to access static member variables
  - Can be called before any class objects are created

Static Member Variables

1) Must be declared in class with keyword **`static`**:

```
class IntVal
{
 public:
    intVal(int val = 0)
    { value = val; valCount++ }
    int getVal();
    void setVal(int);
  private:
    int value;
    static int valCount;
};
```

Static Member Variables

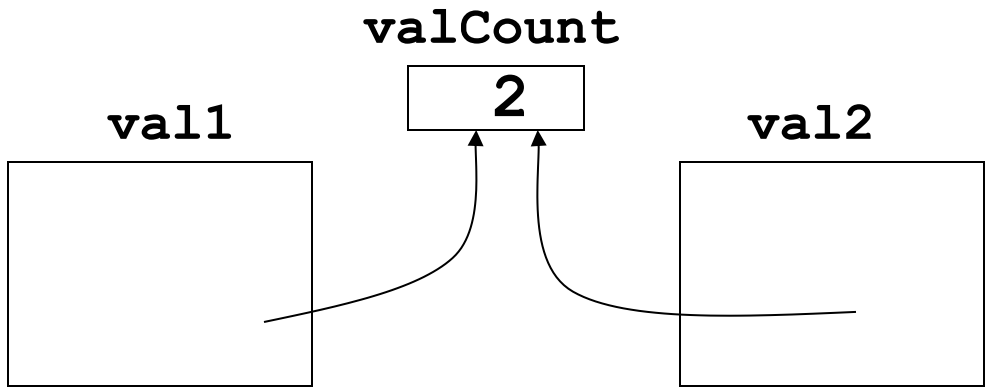
2) Must be defined outside of the class:

```
class IntVal
{
   //In-class declaration
   static int valCount;
   //Other members not shown
};
//Definition outside of class
int IntVal::valCount = 0;
```

Static Member Variables

3) Can be accessed or modified by any object of the class:
   Modifications by one object are visible to all objects of the class:

```
IntVal val1, val2;
```



Static Member Functions

1)Declared with **`static`** before return type:

```
class IntVal
{ public:
    static int getValCount()
   { return valCount; }
  private:
    int value;
    static int valCount;
};
```

Static Member Functions

2) Can be called independently of class objects, through the class name:

```
cout << IntVal::getValCount();
```

3) Because of item 2 above, the **`this`** pointer cannot be used

4) Can be called before any objects of the class have been created

5) Used primarily to manipulate static member variables of the class

11.3 Friends of Classes

- Friend function: a function that is not a member of a class, but has access to private members of the class
- A friend function can be a stand-alone function or a member function of another class
- It is declared a friend of a class with the **`friend`** keyword in the function prototype

Friend Function Declarations

1) Friend function may be a stand-alone function:

```
class aClass
{
  private:
     int x;
     friend void fSet(aClass &c, int a);
};

void fSet(aClass &c, int a)
{
   c.x = a;
}
```

Friend Function Declarations

2) Friend function may be a member of another class:

```
class aClass
{ private:
    int x;
    friend void OtherClass::fSet
                        (aClass &c, int a);
};
class OtherClass
{ public:
     void fSet(aClass &c, int a)
     { c.x = a; }
};
```

Friend Class Declaration

3) An entire class can be declared a friend of a class:

```
class aClass
{private:
   int x;
   friend class frClass;
};
class frClass
{public:
   void fSet(aClass &c,int a){c.x = a;}
   int fGet(aClass c){return c.x;}
};
```

Friend Class Declaration

- If **`frClass`** is a friend of **`aClass`**, then all member functions of **`frClass`** have unrestricted access to all members of **`aClass`**, including the private members.

- In general, restrict the property of Friendship to only those functions that must have access to the private members of a class.

11.4 Memberwise Assignment

- Can use = to assign one object to another, or to initialize an object with an object's data
- Examples (assuming class `V`):

```
V v1, v2;
… // statements that assign
… // values to members of v1
v2 = v1;    // assignment
V v3 = v2;  // initialization
```

11.5 Copy Constructors

- Special constructor used when a newly created object is initialized to the data of another object of same class
- Default copy constructor copies field-to-field, using memberwise assignment
- The default copy constructor works fine in most cases

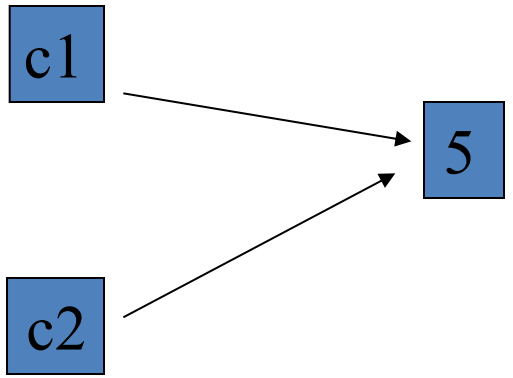
Copy Constructors

Problems occur when objects contain pointers to dynamic storage:

```
class CpClass
{
  private:
    int *p;
  public:
    CpClass(int v=0)
       { p = new int; *p = v;}
    ~CpClass(){delete p;}
};
```

Default Constructor Causes Sharing of Storage

```
CpClass c1(5);
if (true)
{
   CpClass c2=c1;
}
// c1 is corrupted
// when c2 goes
// out of scope
and
// its destructor
// executes
```



Problems of Sharing Dynamic Storage

- Destructor of one object deletes memory still in use by other objects
- Modification of memory by one object affects other objects sharing that memory

Programmer-Defined Copy Constructors

- A copy constructor is one that takes a reference parameter to another object of the same class
- The copy constructor uses the data in the object passed as parameter to initialize the object being created
- Reference parameter should be **`const`** to avoid potential for data corruption

Programmer-Defined Copy Constructors

- The copy constructor avoids problems caused by memory sharing
- Can allocate separate memory to hold new object's dynamic member data
- Can make new object's pointer point to this memory
- Copies the data, not the pointer, from the original object to the new object

Copy Constructor Example

```
class CpClass
{
    int *p;
  public:
    CpClass(const CpClass &obj)
    { p = new int; *p = *obj.p; }
    CpClass(int v=0)
    { p = new int; *p = v; }
    ~CpClass(){delete p;}
};
```

Copy Constructor – When Is It Used?

A copy constructor is called when

- An object is initialized from an object of the same class
- An object is passed by value to a function
- An object is returned using a **`return`** statement from a function

Thanks,..

See you next week isA,..